



Le langage de définition de schéma XML



Le langage de définition de schéma XML

- XML Schéma est une recommandation du W3C de mai 2001 qui consiste à définir une méthodologie de description de document XML autrement que par les DTD.
- L'approche cherche à résoudre plusieurs problèmes des DTD dont l'épine dorsale est le fait qu'un DTD n'est pas un document XML.
- En conclusion, il est nécessaire de connaître un autre langage que XML pour décrire un DTD (le langage propre aux DTD) mais également de disposer d'autres outils pour éditer et manipuler des DTD (par exemple, des parseurs spéciaux).

Le langage de définition de schéma XML

- Un XML Schéma est donc un document dont l'objectif est similaire à un DTD mais sous forme d'un document en XML.
- Plus clairement, c'est un document XML qui décrit un document XML !
- C'est pour cette raison qu'on parle de méta données pour les données constituant un XML Schéma.
- XML Schema est un document XML qui utilise un ensemble de balises définies au sein de la recommandation du W3C.

Le langage de définition de schéma XML

- Le but d'un schéma est de définir une classe de documents XML.
- Il permet de décrire les autorisations d'imbrication et l'ordre d'apparition des éléments et de leurs attributs, tout comme une DTD. Mais il permet aussi d'aller au-delà.
- Un premier point intéressant est qu'un fichier Schema XML est un document XML.
- Cela permet à un tel document d'être manipulé de la même manière que n'importe quel autre fichier XML.

Structure de base

- Comme tout document XML, un Schema XML commence par un prologue, et a un élément racine.

```
<?xml version="1.0" encoding="ISO-8859-1"?>  
<xsd:schema xmlns:xsd="http://www.w3.org/2000/10/XMLSchema">  
  
    <!-- déclarations d'éléments, d'attributs et de types    ici -->  
  
</xsd:schema>
```

Déclarations d'éléments

- Un élément, dans un schéma, se déclare avec la balise **<xsd:element>**.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2000/10/XMLSchema">
    <xsd:element name="contacts" type="typeContacts" />
    <xsd:element name="remarque" type="xsd:string" />
<!-- déclarations de types ici -->
</xsd:schema>
```

Chaque élément déclaré est associé à un type de données via l'attribut `type`. Les éléments pouvant contenir des élément-enfants ou posséder des attributs sont dits de type *complexe*, tandis que les éléments n'en contenant pas sont dits de type *simple*

Déclarations d'attributs

- Contrairement aux éléments, un attribut ne peut être que de type simple.
- Cela signifie que les attributs, comme avec les DTD, ne peuvent contenir d'autres éléments ou attributs.
- De plus, les déclarations d'attributs doivent être placées *après* les définitions des types complexes

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2000/10/XMLSchema">
  <xsd:element name="contacts" type="typeContacts" />
    <xsd:element name="remarque">
      <xsd:complexType>
        <xsd:attribute name="datecom" type="xsd:date" />
      </xsd:complexType>
    </xsd:element>
</xsd:schema>
```

Contraintes d'occurrences

- Tout comme dans une DTD, un attribut peut avoir un indicateur d'occurrences.
- L'élément `attribute` d'un Schema XML peut avoir trois attributs optionnels : `use`, `default` et `fixed`.
- Des combinaisons de ces trois attributs permettent de paramétrer ce qui est acceptable ou non dans le fichier XML final (attribut obligatoire, optionnel, possédant une valeur par défaut...).

```
<xsd:attribute name="datecom" type="xsd:date" use="optional" default="2003-10-11" />
```


Contraintes d'occurrences

- **Déclaration d'une valeur par défaut**

- `<xsd:attribute name="lang" type="xsd:string" default="EN"/>`

- **Déclaration d'une valeur fixe**

- `<xsd:attribute name="lang" type="xsd:string" fixed="EN"/>`

- **Déclaration d'un attribut optionnel**

- `<xsd:attribute name="lang" type="xsd:string" use="optional" />`

- **Déclaration d'un attribut obligatoire**

- `<xsd:attribute name="lang" type="xsd:string" use="required"/>`



Les types de donnée communs

- XML schema a beaucoup de types de données.
Voici la liste des types les plus commun
 - xsd:string
 - xsd:decimal
 - xsd:integer
 - xsd:boolean
 - xsd:date
 - xsd:time

Restriction dans les schémas

- Restriction de la valeur d'élément age entre 0 et 100

```
<xsd:element name="age">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:integer">  
      <xsd:minInclusive value="0"/>  
      <xsd:maxInclusive value="100"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```

Restriction dans les schémas

- Restriction sur l'ensemble des valeurs

```
<xsd:element name="car">
  <xsd:simpleType>
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="Audi"/>
      <xsd:enumeration value="Golf"/>
      <xsd:enumeration value="BMW"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
```

Restriction dans les schémas

- Restriction sur la série des valeurs

```
<xsd:element name="letter">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:string">  
      <xsd:pattern value="[a-z]"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```

Restriction dans les schémas

- Restriction sur la longueur

```
<xsd:element name="password">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:string">  
      <xsd:length value="8"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```

Restriction dans les schémas

- Restriction sur la longueur

```
<xsd:elementname="password">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:string">  
      <xsd:minLength value="5"/>  
      <xsd:maxLength value="8"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```

Élément Complexe

- L'élément complexe est un élément XML qui contient les autres éléments et/ou attributs
- Il y a 4 types d'éléments complexes:
 - Élément vide
 - Élément qui contient seulement d'autres éléments
 - Élément qui contient que des textes, avec éventuellement des attributs.
 - Élément qui contient d'autres éléments et du texte

Définir des types complexes pour des éléments vides

```
<personne ident="2505" />
```

```
<xsd:element name="personne">  
  <xsd:complexType>  
    <xsd:attribute name="ident" type="xsd:ID"/>  
  </xsd:complexType>  
</xsd:element>
```

Définir des types complexes pour des éléments mixtes

<b e="valeur">données

```
<xsd:element name="b">
  <xsd:complexType mixed="true">
    <xsd:attribute name="e" use="required" type="xsd:ID"/>
  </xsd:complexType>
</xsd:element>
```

Définir des types complexes pour un élément

```
<employee>  
  <firstname>Moussa</firstname>  
  <lastname>Bâ</lastname>  
</employee>
```

```
<xsd:element name="employee">  
  <xsd:complexType>  
    <xsd:sequence>  
      <xsd:element name="firstname" type="xsd:string"/>  
      <xsd:element name="lastname" type="xsd:string"/>  
    </xsd:sequence>  
  </xsd:complexType>  
</xsd:element>
```

Personnaliser un type

```
<employee>  
  <firstname>Moussa</firstname>  
  <lastname>Bâ</lastname>  
</employee>
```

```
<xsd:element name="employee" type="personinfo"/>  
<xsd:complexType name="personinfo">  
  <xsd:sequence>  
    <xsd:element name="firstname" type="xsd:string"/>  
    <xsd:element name="lastname" type="xsd:string"/>  
  </xsd:sequence>  
</xsd:complexType>
```

Personnaliser un type

```
<xsd:element name="employee" type="personinfo"/>
<xsd:element name="student" type="personinfo"/>
<xsd:element name="member" type="personinfo"/>

<xsd:complexType name="personinfo">
  <xsd:sequence>
    <xsd:element name="firstname" type="xsd:string"/>
    <xsd:element name="lastname" type="xsd:string"/>
  </xsd:sequence>
</xsd:complexType>
```



Indicateur de types complexes

- On a 7 types d'indicateur
- Indicateurs de l'ordre
 - Tous (All)
 - Choix (Choice)
 - Séquence (Sequence)
- Indicateurs d'occurrence
 - maxOccurs
 - minOccurs

Indicateur All

- Spécifie que des éléments fils peuvent apparaître en n'importe quel ordre et chaque élément fils doit se produire seulement une fois

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:all>
      <xsd:element name="firstname" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:all>
  </xsd:complexType>
</xsd:element>
```

Indicateur Choice

- Spécifie qu'on ne peut avoir qu'un élément fils parmi les éléments fils qui peuvent être produits

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="employee" type="employee"/>
      <xsd:element name="member" type="member"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```


Indicateur Sequence

- Spécifie que les éléments fils doivent être produits en ordre

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="firstname" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Indicateur maxOccurs

- Spécifie le nombre maximum de fois qu'un élément peut être produit

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="full_name" type="xsd:string"/>
      <xsd:element name="child_name" type="xsd:string" maxOccurs="10"/>
```

Indicateur minOccurs

- Spécifie le nombre minimum de fois qu'un élément peut être produit

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="full_name" type="xsd:string"/>
      <xsd:element name="child_name" type="xsd:string"
maxOccurs="10" minOccurs="1"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Autre moyen pour déclarer

```
<xsd:element name="a"> <xsd:complexType>  
  <xsd:sequence maxOccurs="unbounded">  
    <xsd:element ref="b"/>  
    <xsd:element ref="c"/>  
  </xsd:sequence> </xsd:complexType>  
</xsd:element>  
<xsd:element name="b">  
  <xsd:complexType mixed="true">  
    <xsd:attribute name="e" use="required"/>  
  </xsd:complexType>  
</xsd:element>  
<xsd:element name="c" type="xsd:string"/>
```

Définition et utilisation d'un groupe

- Les groupes permettent de déclarer des blocs de sous-éléments.
- **<xsd:group name="PersonneGroup">**
 - **<xsd:sequence>**
 - **<xsd:element name="nom" type="xsd:string"/>**
 - **<xsd:element name="prenom" type="xsd:string"/>**
 - **</xsd:sequence>**
- **</xsd:group>**
- **<xsd:complexType name="MarinType">**
 - **<xsd:sequence>**
 - **<xsd:group ref="PersonneGroup"/>**
 - **<xsd:element name="naissance" type="xsd:date"/>**
 - **</xsd:sequence>**
- **</xsd:complexType>**

Lier le Schéma avec le fichier

- on fait référence au schéma dans le document XML en utilisant l'attribut **xmlns:xsi:noNamespaceSchemaLocation**.
- Par exemple:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<persons xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance  xsi:noNamespaceSchemaLocation="non_fic.xsd">
  <person>
    .....
  </person>
```

Exemple

- Traduire le plus fidèlement possible, ce schéma de base de données en un document XMLSchéma
 - Client (NumCl, Nom, Prénom, Age, Adresse)
 - Commande (NumCo, DateCo, #NumCl)
 - Produit (IdProd, Libellé, Prix, Provenance)
 - LigneCommande (#NumCo, #IdProd, Quantité)